pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Deploying the Service to Vercel

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the service at a public URL

Today

1. What Vercel does with a Python folder
2. vercel_folder, and a GET route for Excel
3. check_deploy_folder, and a planted copy
4. The Hobby plan, as read today
5. Deploying it yourself: two routes
6. The course's own deploy, recorded
7. check_live: yours against bondmath
8. The deploy log, deploys.csv

The notebook stays offline

Every cloud step ran once on the course's account and was recorded, masked. You run the same steps on yours.

Day 6 ran the service at 127.0.0.1. Today it gets an address anyone can reach.

What the service returns, and where it may run

Numbers in, numbers out	units in every key: price_per_100, dv01_per_100
Personal, non-commercial	Hobby is for a course project on your own account
Firm policy first	a desk tool at work goes through your firm's process

The service's outputs

Allowed names

price per 100 of face

DV01 per 100 of face

the service returned

Never called

fair value, cheap, or rich

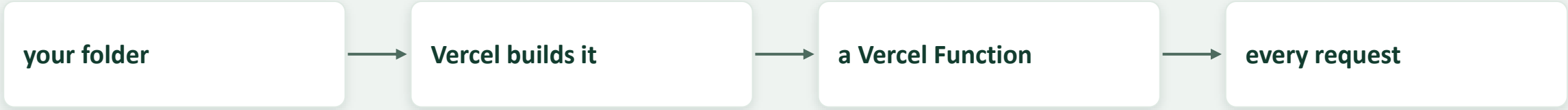
a quote or a mark

a recommendation

How Vercel runs a Python folder

The problem	A public address for the service, with no server to start, stop, or patch
The method	Vercel reads requirements.txt, installs what it lists, loads app from the entrypoint
A request	Vercel starts the function if it is not running, hands it the request, returns the answer
You control	The folder: requirements, entrypoint, and the Python version
What comes out	A public address that answers requests

What Vercel does with a Python folder



`dependencies`	requirements.txt naming the framework (FastAPI), pinned
`entrypoint`	app.py, index.py, server.py, main.py, wsgi.py, or asgi.py, defining app
`Python`	3.12 unless the folder asks for 3.13 or 3.14 in .python-version
`bundle`	at most 500 MB uncompressed, everything the function holds

No configuration file: Vercel finds FastAPI in requirements.txt and app in the entrypoint.

Vercel Python: 3.12 by default, 3.13 by .python-version, 500 MB a bundle (vercel.com/docs/functions/runtimes/python, read 2026-10-12).

How vercel_folder works

The problem	A deploy folder should hold what the service needs and nothing that happened to be lying around
The method	It writes app.py, requirements.txt (one pinned line), .python-version, and README.md
The modules	Copied beside them unchanged: the tested code is the deployed code
You control	The target folder and the list of modules
What comes out	A folder safe to send to the platform or to a public repository

The deploy folder, built by code

```
.python-version          5 bytes
README.md              421 bytes
app.py                 117 bytes
bondmath.py         30,916 bytes
deploykit.py        36,037 bytes
requirements.txt        17 bytes
```

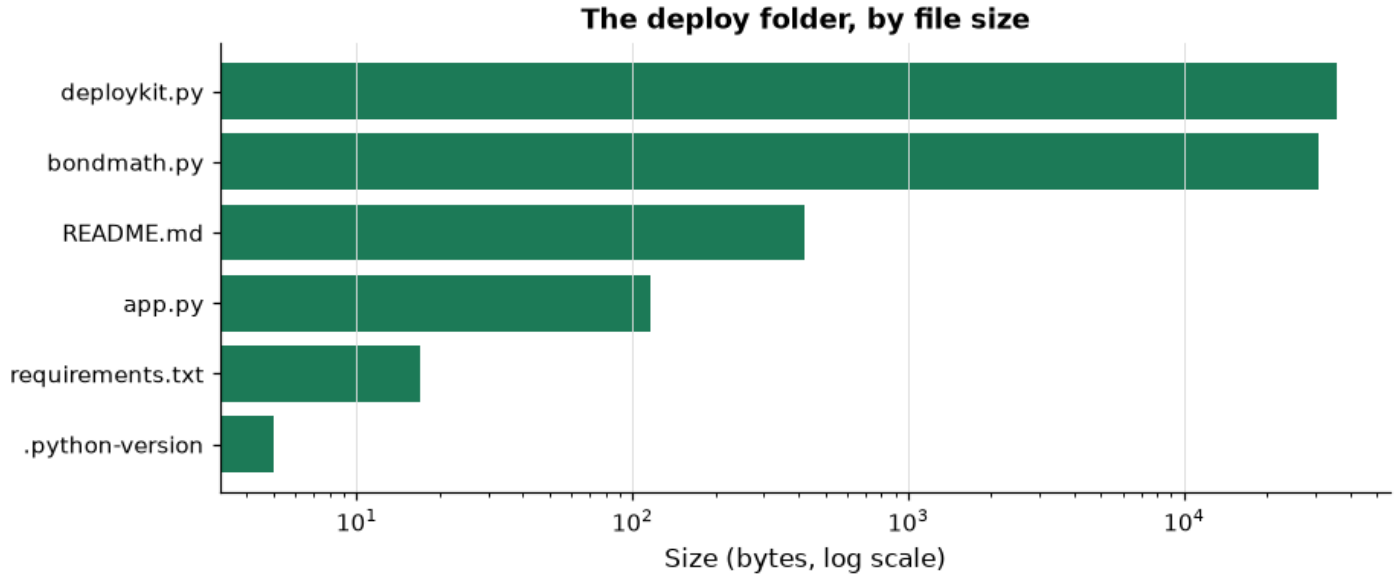
What Vercel reads

`requirements.txt`	fastapi, pinned: one line
`.python-version`	3.13
`app.py`	app = deploykit.bond_app()
`README.md`	what it does, and the disclaimer

Plus a GET route for Excel, appended to app.py: WEBSERVICE sends GET only

```
POST /dv01: status 200, {'dv01_per_100': 0.043624879198739 52}
GET  /dv01?settlement=2026-10-02&maturity=2031-10-02&coupon_pct=5&yield_pct=5.06&frequency=2
           status 200, {'dv01_per_100': 0.043624879198739 52}
the two routes agree to the last digit: True
```

The deploy folder, by size



File	Bytes
.python-version	5
README.md	421
app.py	117
bondmath.py	30,916
deploykit.py	36,037
requirements.txt	17

The two modules are the weight; what Vercel reads to know what to do is a few lines.

How check_deploy_folder works

The problem	Everything in the folder goes to a build machine and to a public repository
The method	It runs public_scan, reads the pins, looks for an entrypoint that defines app, measures the size
The report	One sentence per problem: file, line, and kind, never the value found
You control	The folder and the size limit
What comes out	A list of problems; an empty list means none were found

Check the folder before it leaves

no .env file	no key	pinned lines	app defined	under 500 MB

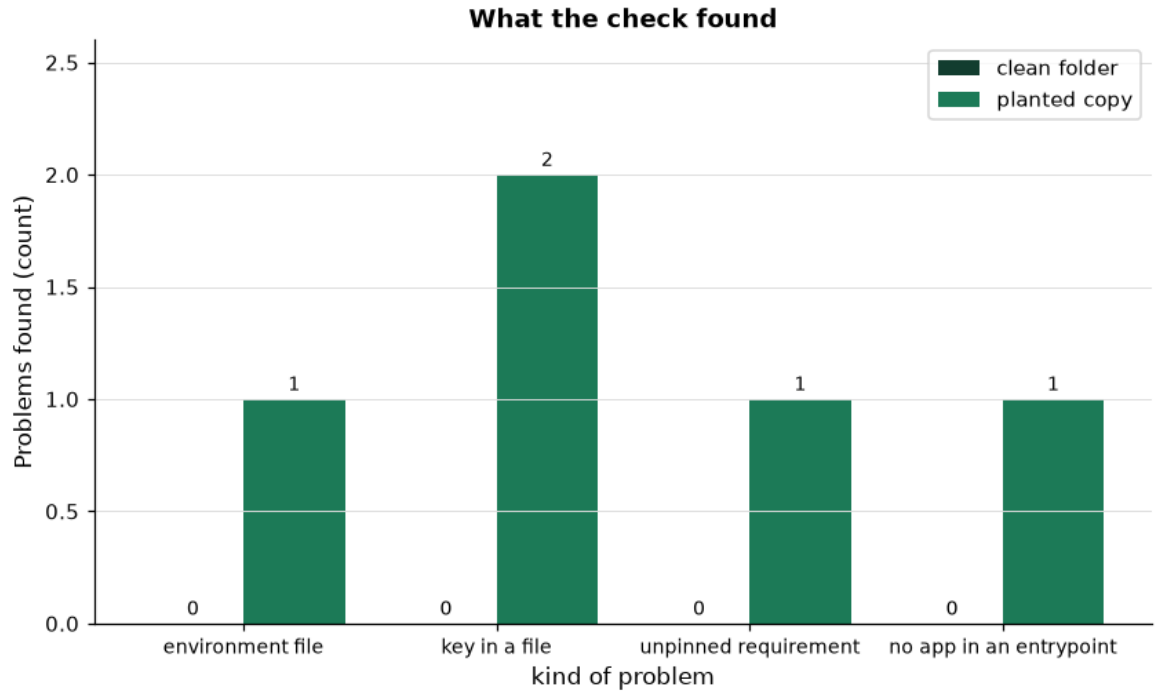
```
problems: 0; the folder is 0.07 MB (MB = 1024 x 1024 bytes), against Vercel's 500 MB
```

A planted copy, four plants:

```
config/.env.production: an environment file at any depth is never deployed
config/.env.production, line 1: a key-like string (sk-or- pattern, starts 'sk-', 18 characters)
config.json, line 1: a key-like string (sk-or- pattern, starts 'sk-', 18 characters)
requirements.txt line 2: pandas is not pinned with ==
no entrypoint defines app (looked in app.py, index.py, server.py, main.py, wsgi.py, asgi.py, and ...
problems: 5; safe to deploy: no
```

A key is named by its kind, first three characters, and length. Never its value.

What the check found



Kind of problem	Clean	Planted
environment file	0	1
key in a file	0	2
unpinned requirement	0	1
no app in an entrypoint	0	1

Five problems from four plants, each found without running the app.

The Hobby plan, as read today

1,000,000

function invocations a month

4

active CPU hours a month

100

deployments a day

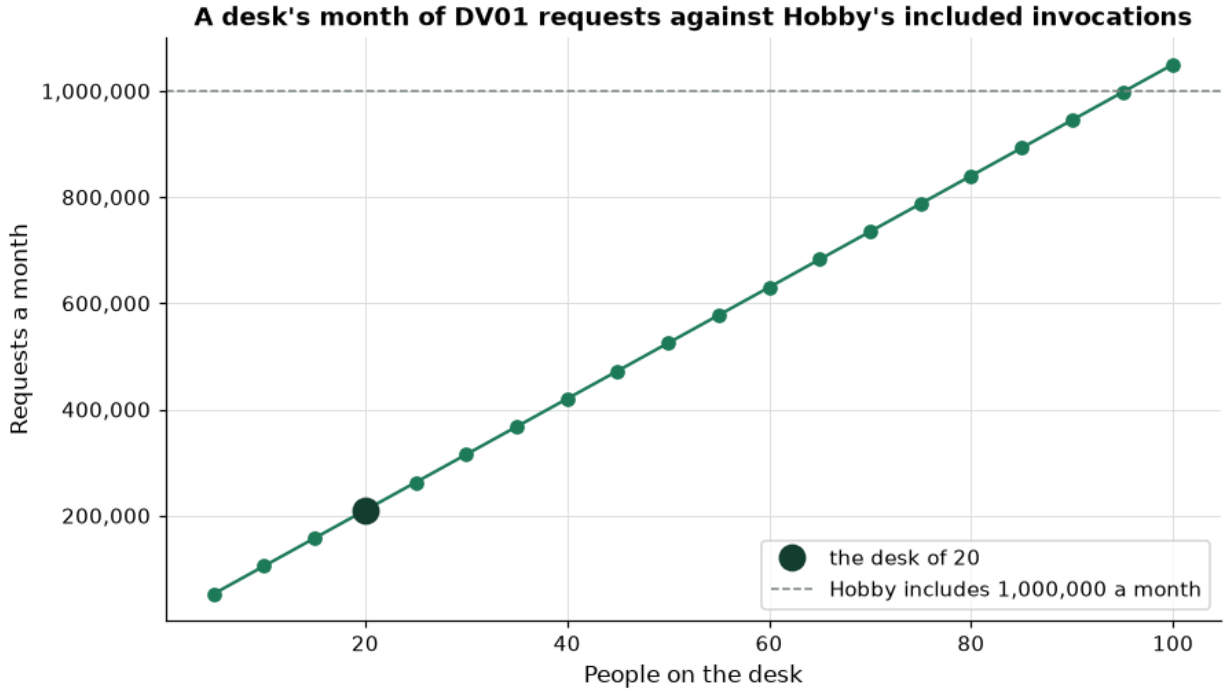
```
the desk's month: 210,000 requests, 21% of the invocations included
```

"the Hobby plan restricts users to non-commercial, personal use only": the numbers fit a desk; the terms do not.

Vercel Hobby: 1,000,000 invocations and 4 CPU hours a month; non-commercial use only (vercel.com/docs/plans/hobby, read 2026-10-12).

Vercel Hobby: 100 deployments a day, 200 projects, 300 s a request (vercel.com/docs/plans/hobby, read 2026-10-12).

A desk's month against Hobby



People	Requests	Share (%)
5	52,500	5.2
20	210,000	21.0
50	525,000	52.5
100	1,050,000	105.0

Arithmetic only: Hobby is for non-commercial, personal use, whatever the size of the desk.

Two routes to your own deploy

Route 1: GitHub import (no Node.js)

check_deploy_folder: []

push to a public repo, pfi-bond-service

Vercel: New Project, the repo, Deploy

push to main: a new production deploy

a pull request: a preview URL

Route 2: the CLI (needs Node.js)

```
npm i -g vercel
vercel login
vercel deploy --prod --yes
```

how the course's own deploy was recorded

Then MY_SERVICE_URL= your production domain, in my_bondmath\.env. Preview addresses ask for a sign-in.

How a deploy works

The problem	A folder on your machine has to become an address anyone can reach
The upload	The CLI sends the folder's files to Vercel
The build	Vercel's machine installs the requirements, compiles the bytecode, stores the result
The address	The production address goes to the new deployment: Aliased
What comes out	A deployment with its own page, a stamped build log, and the project's address pointing at it

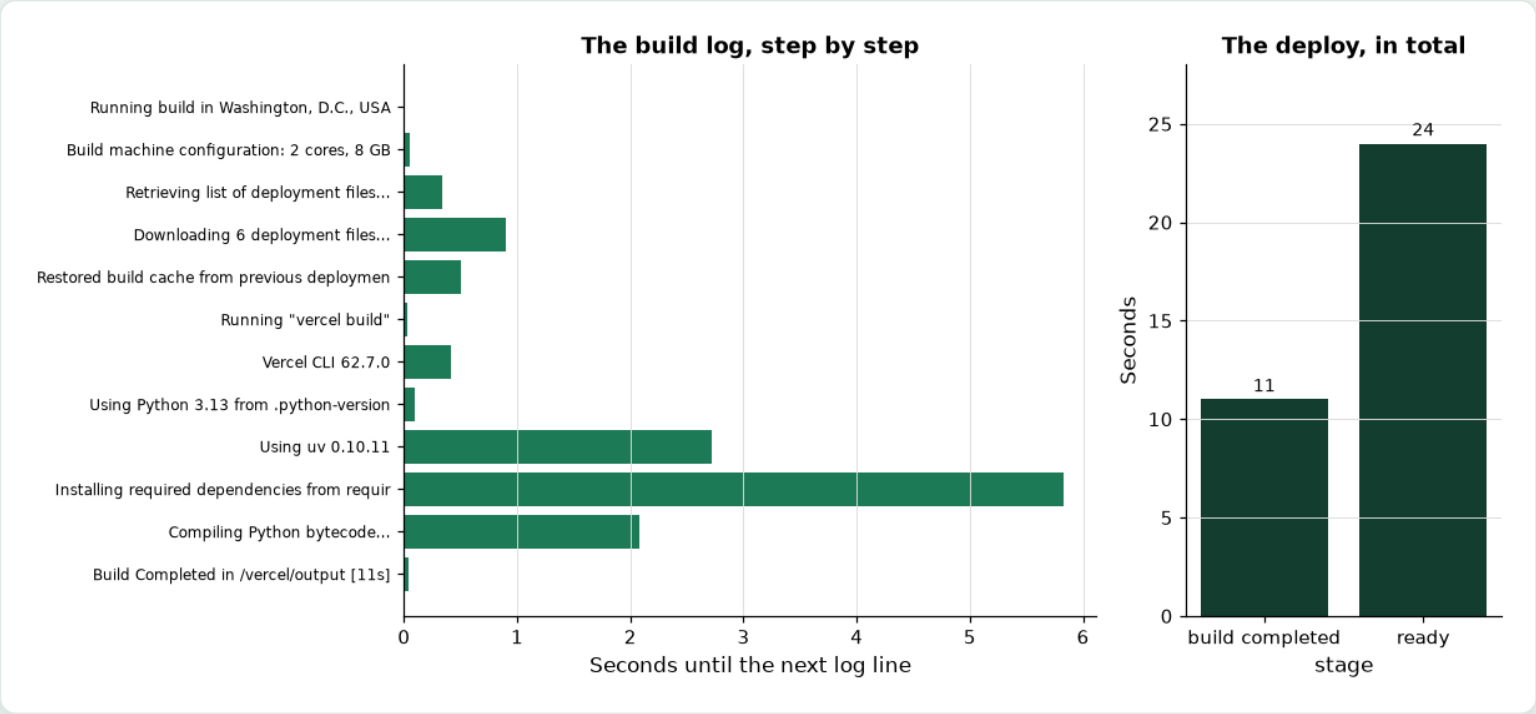
The course's own deploy

What the platform printed, masked

```
Recorded 2026-10-12 on the course's own account; account names, URLs, and ids masked to `student`.
> vercel deploy --prod --yes
  Directory       ~\projects\pfi-bond-service
? Linked          student/pfi-bond-service
Downloading 6 deployment files...
Using Python 3.13 from .python-version
Installing required dependencies from requirements.txt...
Build Completed in /vercel/output [11s]
? Aliased         https://student.vercel.app
? Ready in 24s
```

Built in 11 s on Vercel's machine; ready in 24 s. Every address masked to student.vercel.app.

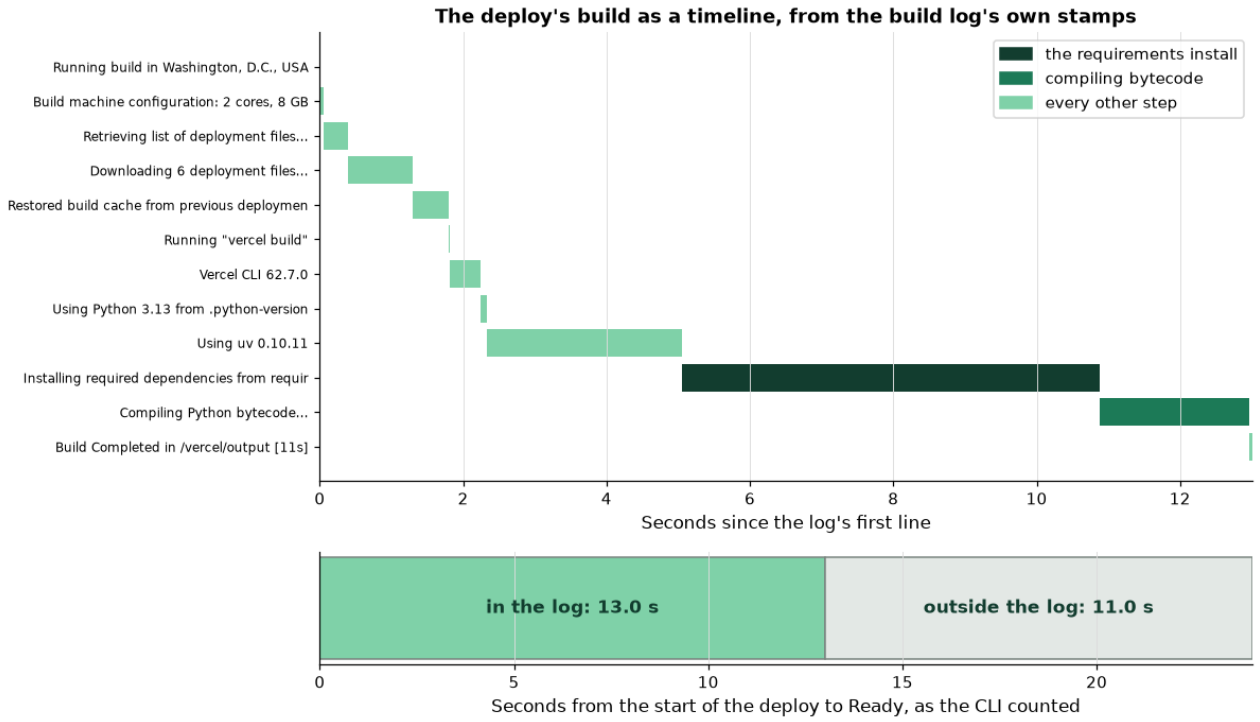
Where the build's time went



Longest steps	Seconds
Installing required depend	5.8
Using uv 0.10.11	2.7
Compiling Python bytecode.	2.1
Downloading 6 deployment f	0.9
Restored build cache from	0.5

Built in 11 s on Vercel's machine; ready in 24 s. One deploy, one day: yours will differ.

The build as a timeline



Step	From s	Lasts
Installing required de	5.1	5.8
Compiling Python bytec	10.9	2.1

The log spans 13.0 s of the 24 s to Ready; 11.0 s are outside the log

Three requests, and cold starts

```
curl.exe -s -w "`nHTTP status %{http_code}`n" https://student.vercel.app/health
{"status":"ok","version":"1.0","as_of":null}
POST /dv01 (Invoke-WebRequest):  {"dv01_per_100":0.04362487919873952}
GET /dv01?settlement=...&frequency=2:  {"dv01_per_100":0.04362487919873952}
```

A cold start

A Vercel Function is not running between requests: the first request after a quiet spell waits while Vercel starts the app.

Vercel Fluid compute: cold starts reduced by bytecode optimization and pre-warming (vercel.com/docs/fluid-compute, read 2026-10-12).

How check_live works

The problem	A deployment needs a test against one trusted answer: bondmath on your machine
The method	One POST for each case to your address; the same endpoint computed locally
The table	Local value, deployed value, and whether they are exactly equal
You control	The address and the cases
What comes out	A row for each case; a status other than 200 stops it, naming the endpoint

check_live: your deployment against bondmath

live-deploy cell	your MY_SERVICE_URL; skipped in the saved notebook
offline	the folder's app through the test client
recorded	the course's deployment, from PowerShell

```
/price    here 99.7378446367698  deployed 99.7378446367698  equal: True
/accrued  here 0.0  deployed 0.0  equal: True
/dv01     here 0.04362487919873952  deployed 0.04362487919873952  equal: True
/yield    here 5.060000000023592  deployed 5.060000000023592  equal: True
every endpoint equal to bondmath here: True
```

The folder's copy of deploykit.py was stale: before the copy no, after it yes. Refresh it before every deploy.

The deploy log, and Excel

```
date 2026-10-12
target vercel production
url_masked https://student.vercel.app
commit <the folder's commit>
tests_passed 30
bundle_mb 0.07
status ready
```

```
A1  =WEBSERVICE("https://student.vercel.app/health")
A2  =WEBSERVICE("https://student.vercel.app/dv01?settlement=2026-10-02&...")
A3  =VALUE(TEXTBEFORE(TEXTAFTER(A2,"""dv01_per_100"":"),"}"))
A4  =WEBSERVICE("https://student.vercel.app/dv01?settlement=2031-10-02&...")
```

A1 `{"status":"ok","version":"1.0","as_of":null}`

A2 `{"dv01_per_100":0.04362487919873952}`

A3 `0.0436248791987395`

A4 `#VALUE!  (maturity before settlement: a 422)`

Tested in Excel 16.0 on the course's deployment: A3 equals the service at 15 significant digits.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: rglob("*.py"): the check reads only Python files, so a key in .env.production or config.json ships.

Code that runs is not the same as code that is right.

What you can do now

- Vercel: requirements.txt, an app, a function
- Build the deploy folder with code
- Check it before it leaves
- Hobby is personal and non-commercial
- check_live, and a row in deploys.csv

Week 2, Day 3 The credit desk as a service